Probability Basics (I)

- probability :

probability denotes the possibility of the outcome of any random event.

event :

event is an outcome or set of outcomes resulting from an experiment.

sample space :

(Ω) The set of all possible outcomes is known as the sample space. Therefore event is also known as a subset of the sample space.

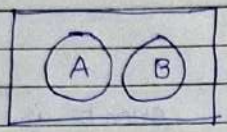
empty set : $\emptyset$, $\{\}$

impossible event :

whose set of outcome is an empty set.

mutually exclusive events :
(disjoint events)

events that do not occur at the same time.



$P(A \cap B) = 0$, $P(A \cup B) = P(A) + P(B)$

Independent events :

If the probability of occurence of an event is not affected by the occurance of another event, then ~~is~~ it is said to be independent events.

$P(A \cap B) = P(A) \cdot P(B)$

$P(A|B) = P(A)$

$P(B|A) = P(B)$

$P(\cap_{i \in I} A_i) = \prod_{i \in I} P(A_i)$

- From the above, It should clear that pair-wise independance does not imply independence.

Exhaustive events :

A set of events is said to be exhaustive if at least one of them surely occurs whenever the experiment is executed.

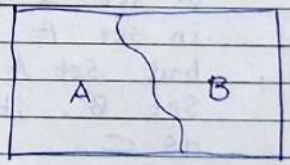
Complementary events :

two events are said to be complementary if, one event takes place if and only if the other event does not take place.

$P(A) + P(B) = 1$

$P(A \cup B) = P(A) + P(B) = 1$

$P(A \cap B) = 0$



- Sample space is denoted by Ω. and individual elements (outcomes) are denoted by ω.

Ω (Capital Omega), ω (Small Omega)

- Event is subset of Ω.

- Subset : ~~for~~

for A and B, set A is subset of set B if every element in set A is also in Set B. it is written as $\subseteq$.

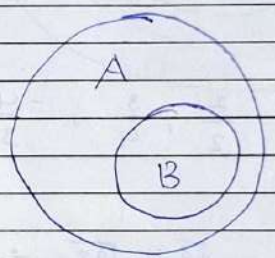
- proper subset :

Set A is a proper subset of set B if every element in set A is also in set B, but set A does not equal Set B. it is written as $\subset$.

Superset : $X \supseteq Y$, where X may or may not be equal to Y. it is denoted by $\supseteq$.

proper superset :

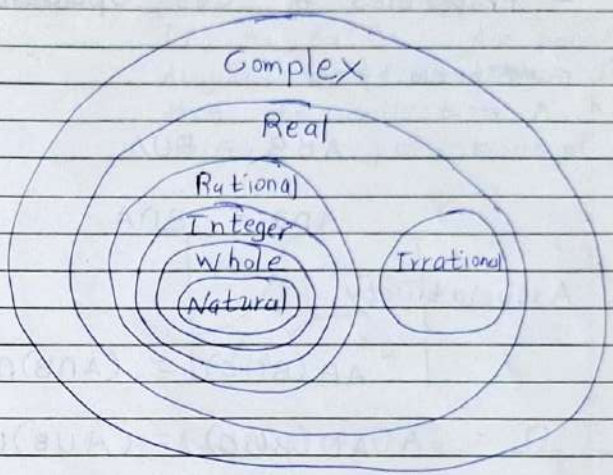
- $X \supset Y$ but X is strictly not equal to Y.
- it is denoted by $\supset$.



- A is superset of B.
- B is subset of A.

Types of Numbers:

- Natural (N) : 1, 2, 3, ...
- Whole (W) : 0, 1, 2, 3, ...
- Integer (Z) : ... -3, -2, -1, 0, 1, 2, 3 ...
- Rational (Q) : $\frac{1}{2}, \frac{3}{4}, \frac{-4}{3}, \frac{4}{1}$ ...
- Irrational (P) : $\sqrt{3}, \sqrt{91}, -\sqrt{44}$ ...
- Real (R) : Includes all of above
- Complex (C) : $P + iq$



- Properties of set operations :

Commutativity

$$A \cup B = B \cup A$$

$$A \cap B = B \cap A$$

Associativity

$$A \cap (B \cap C) = (A \cap B) \cap C$$

$$A \cup (B \cup C) = (A \cup B) \cup C$$

Distributivity

$$A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$$

$$A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$$

DeMorgan's Law

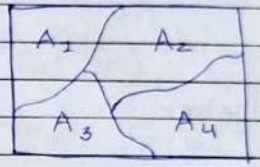
$$(A \cup B)^c = A^c \cap B^c$$

c = complimantory

$$(A \cap B)^c = A^c \cup B^c$$

Partition :

If $A_1, A_2, A_3 \ldots$ are pair-wise disjoint and $\bigcup_{i=1}^{\infty} A_i = \Omega$, then the collection $A_1, A_2 \ldots$ forms a partition of Ω.



$$A_1 \cup A_2 \cup A_3 \cup A_4 = \Omega$$

Power set :

the power set of set A is defined as the set of all subsets of the set A including the set itself and the null or empty set it is denoted by P(A).

$$A = \{1, 2, 3\}$$

$$\Rightarrow P(A) = \{\emptyset, \{1\}, \{2\}, \{3\}, \{1,2\}, \{2,3\}, \{1,3\}, \{1,2,3\}\}$$

or 2^A

- if A has n elements, then P(A) has 2^n elements. P(A) is also denoted as 2^A.

Sigma algebra :

A sigma algebra is a set F of subsets of Ω, with the following properties:

(a) $\phi \in F$
(b) If $A \in F$, then $A^c \in F$
(c) If $A_i \in F$ for every $i \in N$, then $\bigcup_{i=1}^{\infty} A_i \in F$

- F is subset of Power set of A P(A).

$$F \subseteq P(A)$$

- all sets that belongs to F is called or F - measurable set (event).

- Largest sigma Algebra, $F_{max} = P(A)$ (Power of set A)

- Smallest Sigma Algebra, $F_{min} = \{\emptyset, A\}$

- ex. $A = \{a, b, c, d\}$

$F_1 = \{\emptyset, A\}$

$F_2 = \{\emptyset, \{a, b\}, \{c, d\}, \{a, b, c, d\}\}$

$\vdots$

$F_n = P(A)$

- If Ω is countable, then all the events (Power set elements) are measurable set.

but this can not be applied ~~to all~~ when Ω is uncountable.

Bonferroni's Inequality :

$$P(A \cap B) \geq P(A) + P(B) - 1$$

General form:

$$P(\cap_{i=1}^{n} A_i) \geq \sum_{i=1}^{n} P(A_i) - (n-1)$$

- gives a lower bound on the intersection probability which is useful when this probability is hard to calculate.
- Only useful if the probabilities of individual events are sufficiently large.

Boole's Inequality :

$$P(\cup_{i=1}^{\infty} A_i) \leq \sum_{i=1}^{\infty} P(A_i)$$

- gives a useful upper bound for the probability of the union of events.

$$P(A \cup B) + P(A \cap B) = P(A) + P(B)$$

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}$$

Bayes' Rule :

$$P(A \mid B) = \frac{P(B \mid A) \cdot P(A)}{P(B)}$$

$$P(A_i \mid B) = \frac{P(B \mid A_i) \cdot P(A_i)}{P(B)},$$

$$P(B) = \sum_{i=1}^{\infty} P(B \mid A_i) P(A_i)$$

$A_1, A_2 \ldots$ be partition of the sample space, and B be any subset of the sample space.

Conditional independence:

Let A, B and C be three events with $P(C) > 0$. The events A and B are called conditionally independent given C if

$$P(A \cap B \mid C) = P(A \mid C) \cdot P(B \mid C)$$

or equivalently

$$P(A \mid B \cap C) = P(A \mid C)$$

Random variable:

A random variable is a function $X : \Omega \to R$.

It is a function that gives a value in number to each element of a sample space.

The variable used to measure the outcome of the random experiment is called a random variable.

Example: let, X be the sum of outcomes on rolling 3 dice.

$\Omega = \{HHH, HHT, HTH, HTT, THH, THT, TTH, TTT\}$

$\Omega = \{\omega_1, \omega_2, \omega_3 \ldots\}$

to creat this Ω, we have also used Random variable. Actually, for creation of any sample space or set of outcomes we need a Random variable.

INDUCED PROB FUNCTION

Consider the example experiment of tossing a fair coin 3 times. Let X be the numbe of heads obtained in the three tosses. Enumerating the elementary outcomes, we observe the value of X as

ω	HHH	HHT	HTH	THH	TTH	THT	HTT	TTT
$X(\omega)$	3	2	2	2	1	1	1	0

{ X is a random variable, random function, creator of the new set of outcomes }

New set of outcomes = {0, 1, 2, 3}

Probability of the random variable in its range :

	X	0	1	2	3
Induced Probability Function	$P_X(X=x)$	1/8	3/8	3/8	1/8

Induced probability function which maps each possible value of the random variable to its associated probability value.

- Let $\Omega = \{\omega_1, \omega_2, \ldots\}$ be a sample space and P be a probability measure (function).

Let X be a random variable with range $\mathcal{X} = \{x_1, x_2, \ldots\}$

We define the induced probability function P_X on $\mathcal{X}$ as

$$P_X(X = x_i) = P(\{\omega_j \in \Omega : X(\omega_j) = x_i\})$$

Cumulative Distribution Function :

The cumulative distribution function or cdf of a random variable X, denoted by $F_X(x)$, is defined by

$$F_X(x) = P_X(X \leq x), \text{ for all } x$$

Example :

x	$(-\infty, 0]$	$(-\infty, 1]$	$(-\infty, 2]$	$(-\infty, 3]$	$(-\infty \infty)$
$F_X(x)$	1/8	1/2	7/8	1	1

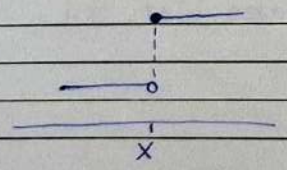
or

$P_X(X \leq x)$

- If $x \leq y$, then $F_X(x) \leq F_X(y)$
- $\lim_{x \to -\infty} F_X(x) = 0$, $\lim_{x \to \infty} F_X(x) = 1$
- $\lim_{y \downarrow x} F_X(y) = F_X(x)$

$y \downarrow x = y \to x+$ (decreasing)

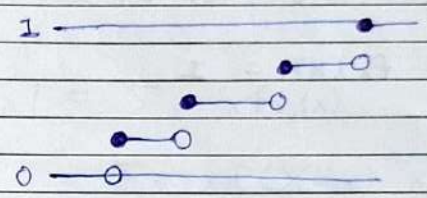
$y \uparrow x = y \to x-$ (increasing)

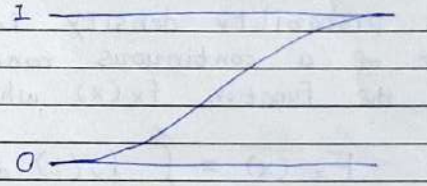


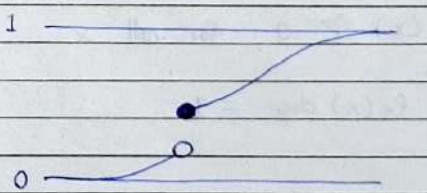
Continuous and Discrete Random Variables

A random variable X is continuous if $F_X(x)$ is a continuous function of x.

A random variable X is discrete if $F_X(x)$ is a step function of x.







Probability Mass Function :

The probability mass function or pmf of a discrete random variable X is given by

$$f_X(x) = P(X = x), \text{ for all } x$$

- $f_X(x) \geq 0$, for all x
- $\sum_x f_X(x) = 1$

Probability density function :

The probability density function or pdf of a continuous random variable is the function $f_X(x)$ which satisfies

$$F_X(x) = \int_{-\infty}^{x} f_X(t)\, dt, \text{ for all } x$$

- $f_X(x) \geq 0$, for all x
- $\int_{-\infty}^{\infty} f_X(x)\, dx = 1$

Expectation :

The expected value or mean of a random variable X, denoted by $E[X]$, is given by

$$E[X] = \int_{-\infty}^{\infty} x f_X(x)\, dx \quad \text{(continuous RV)}$$

$$E[X] = \sum_x x f_X(x)$$

$$= \sum_x x P(X = x) \quad \text{(discrete RV)}$$

Q Let the random variable X take values -2, -1, 1, 3 with probabilities 1/4, 1/8, 1/4, 3/8 respectively. what is the expectation of the random variable $Y = X^2$?

→ The random variable Y takes on the values 1, 4, 9 with probabilities 3/8, 1/4, 3/8 respectively.

Hence,

$$E(Y) = \sum_x x P(Y = x)$$

$$= 1 \cdot \frac{3}{8} + 4 \cdot \frac{1}{4} + 9 \cdot \frac{3}{8}$$

$$= \frac{19}{4}$$

$$E(Y) = E(X^2) = \sum_x x^2 P(X = x)$$

$$= 4 \cdot \frac{1}{4} + 1 \cdot \frac{1}{8} + 1 \cdot \frac{1}{4} + 9 \cdot \frac{3}{8}$$

$$= \frac{19}{4}$$

Example:

Let, X is a random variable and $Y = g(X)$, then Y itself is a random variable.

range of Y can be written as

$$R_Y = \{ g(x) : x \in R_X \}$$

If we already know the PMF of X, to find the PMF of $Y = g(X)$, we can write

$$P_Y(y) = P(Y = y)$$

$$= P(g(X) = y)$$

$$= \sum_{x : g(x) = y} P_X(x)$$

If $Y = X^2 + 2$,

$$P_Y(y) = P(Y = y)$$

$$= P(g(X) = y)$$

$$= P(X^2 + 2 = y)$$

$$= P(x = \pm\sqrt{y-2})$$

Properties of Expectations :

Let X be a random variable and let a, b, c be constants. Then, for functions $g_1(X)$ and $g_2(X)$ whose expectations exist

$$E(a g_1(X) + b g_2(X) + c) = a E g_1(X) + b E g_2(X) + c$$

If $g_1(X) \geq 0$ for all x, then $E g_1(X) \geq 0$

If $g_1(X) \geq g_2(X)$ for all x, then

$$E g_1(X) \geq E g_2(X)$$

If $a \leq g_1(X) \leq b$, for all x, then

$$a \leq E g_1(X) \leq b$$

Moments :

for each integer n, the n^{th} moment of X is

$$\mu_n' = E(X^n)$$

The n^{th} central moment of X is

$$\mu_n = E((X-\mu)^n)$$

- first moment, $\mu' = E(X)$ = mean of the X

- second central moment, $\mu_2 = E((X-\mu)^2)$ which is varionce of the X.

$$\therefore \text{Var}X = E((X-\mu)^2)$$

$$= E((X - EX)^2)$$

$$= E(X^2 - 2XEX + (EX)^2)$$

$$= EX^2 - 2EX \cdot EX + (EX)^2$$

$$\therefore \text{Var}X = EX^2 - (EX)^2$$

$\{ \because E(k) = k \}$

The Positive square root of $Var X$ is the standard deviation of X.

Note :

$$Var(aX + b) = a^2 Var X$$

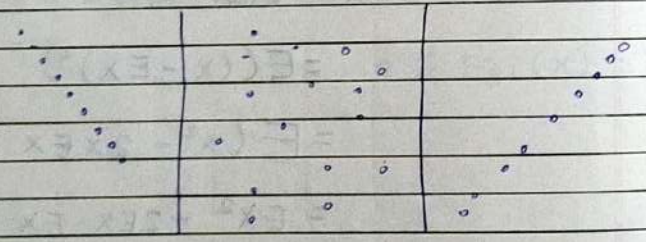
where, a, b are constants

Covariance :

The covariance of two random variables, X and Y is

$$cov(X, Y) = E[(X - EX)(Y - EY)]$$
$$= \sigma_{xy}$$

It is a measure of how two random variables change together.



Large Negative | Near Zero | Large Positive

Correlation :

The correlation of two random variables, X and Y is

$$\rho(X, Y) = \frac{cov(X, Y)}{\sqrt{var X \cdot var Y}}$$
$$= \frac{\sigma_{xy}}{\sigma_x \cdot \sigma_y}$$

$\rho(X, Y)$ lies between -1 and $+1$

$$-1 \leq \rho \leq 1$$

Joint Probability Mass Function (PMF):

$$f_{X,Y}(x,y) = P(X=x, Y=y)$$

from this joint PMF we can obtain the PMF's of the two random variables

range of f_{XY}

$$R_{XY} = R_X \times R_Y$$

$$f_X = \sum_y f_{X,Y}(x,y)$$ (marginal PMF of R.V. X)

$$f_Y = \sum_x f_{X,Y}(x,y)$$ (marginal PMF of R.V. Y)

$$\sum_{(x_i,y_i) \in R_{XY}} P_{XY}(x_i, y_i) = 1$$

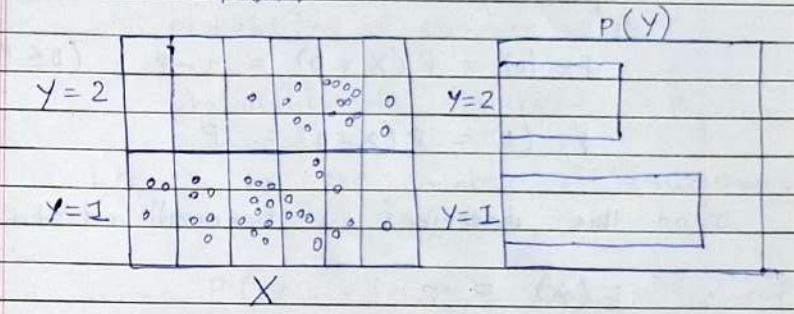
$$f_{X,Y}(x,y) = P(X=x, Y=y)$$

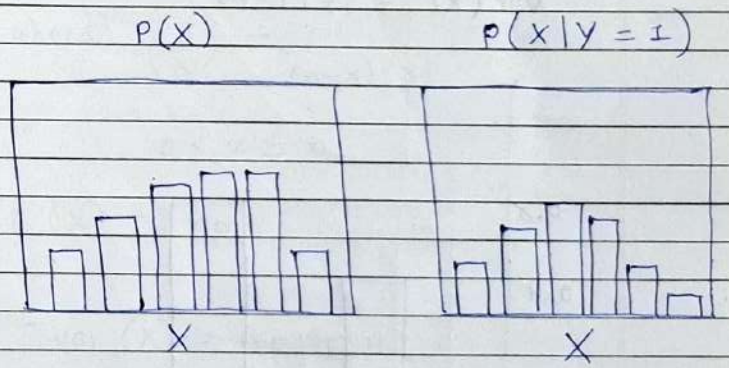
$$= P((X=x) \cap (Y=y))$$

Conditional Distributions:

$$f_{X|Y}(x|y) = P(X=x \mid Y=y)$$

$$= f_{X,Y}(x,y) / f_Y(y)$$





Bernoulli Distribution :

Consider a random variable X taking one of two possible values (either 0 or 1). Let the PMF of X be given by

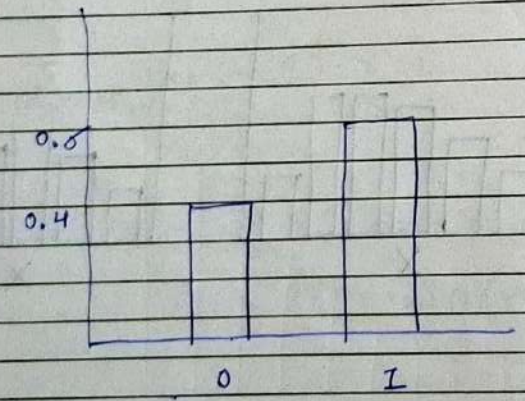
$f_X(0) = P(X=0) = 1-p \quad (0 \le p \le 1)$

$f_X(1) = P(X=1) = p$

This describes a Bernoulli distribution

$E(X) = p$

$var(X) = p(1-p)$



Binomial Distribution

Consider the situation where we perform n independent Bernoulli trials where

probability of success = p

probability of failure = 1 - p

Let X be the number of successes in the n trials, then we have

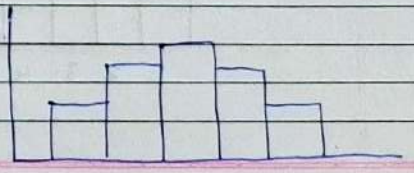
$$P(X = x \mid n, p) = \binom{n}{x} p^x (1-p)^{n-x}$$

where $\binom{n}{x} = \frac{n!}{(n-x)!\,x!}$

$0 < x \le n$

$E(X) = np$

$var(X) = np(1-p)$



Geometric Distribution :

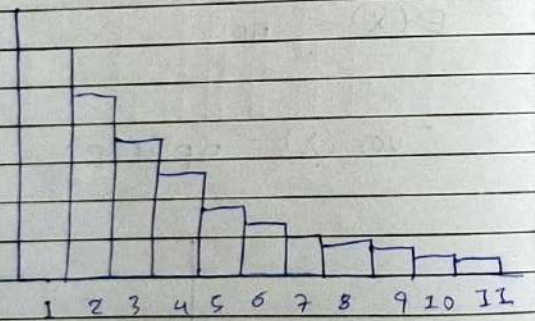
Suppose we perform a series of independent Bernoulli trials, each with a probability p of success. Let X represent the number of trials required for first success, then we have

$$P(X = x \mid p) = (1-p)^{x-1} p$$

$$X = 1, 2, 3 \ldots$$

$$E(X) = \frac{1}{p}$$

$$Var(X) = \frac{(1-p)}{p^2}$$



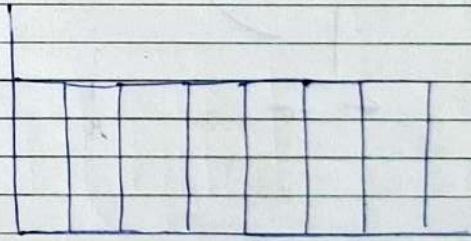
Uniform Distribution :

A continuous random variable X is said to be uniformly distributed on an interval [a, b] if its PDF is given by

$$f_X(x \mid a, b) = \begin{cases} \frac{1}{b-a} & ; \text{if } X \in [a, b] \\ 0 & \text{otherwise} \end{cases}$$

$$E(X) = \frac{(a+b)}{2}$$

$$Var(X) = \frac{(b-a)^2}{12}$$

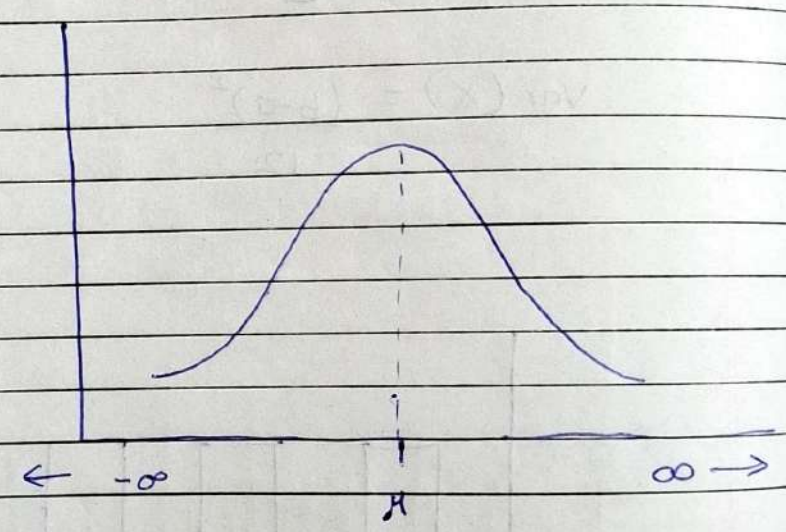


Normal Distribution :
(Gaussian Distribution)

A continuous random variable X is said to be normally distributed with parameters μ and σ^2 if the density of X is given by

$$f_X(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

$$-\infty < x < \infty$$



Beta Distribution :

The pdf of the beta distribution in the range $[0, 1]$, with shape parameters α, β, is given by

$$f(x \mid \alpha, \beta) = \frac{\Gamma(\alpha+\beta)}{\Gamma(\alpha)\cdot\Gamma(\beta)} x^{\alpha-1} (1-x)^{\beta-1}$$

where the gamma function is an extension of the factorial function.

$$E(X) = \frac{\alpha}{\alpha+\beta}$$

$$\text{var}(X) = \frac{\alpha\beta}{(\alpha+\beta)^2(\alpha+\beta+1)}$$

Norm :

lp norm

$$\|x\|_p = \left(\sum_{i=1}^{n} |x_i|^p\right)^{\frac{1}{p}}$$

l2 norm is magnitude of vector.

- Matrices can have norms too

Frobenius norm

$$\|A\|_F = \sqrt{\sum_{i=1}^{m}\sum_{j=1}^{n} A_{ij}^2}$$

$$= \sqrt{tr(A^T A)}$$

(square of all ~~not~~ elements and sum it)

Inductive and Deductive Machine Learning :

Inductive ML learns from specific examples to derive general principles, while deductive ML starts with general principles to make decisions about specific cases.

What is Machine Learning ?

ML is said to learn from experience with respect to some class of tasks, and a performance measure P, ~~if~~ The learner's performance at tasks in the class, as measured by P, improves with experience.

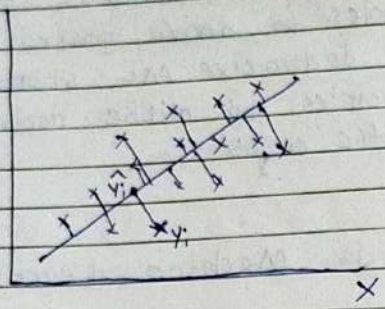
Inductive Learning

Types of Machine learing

1) Supervised ML
2) Unsupervised ML
3) Reinforcement ML

Simple Linear Regression (OLS)

→ OLS method :



$$\hat{Y} = \theta_0 + \theta_1 x$$

we have to find θ_0 and θ_1.

$$J(\theta_0, \theta_1) = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

Here, $J(\theta_0, \theta_1)$ is cost function. (error function)

y_i is actual output.

$\hat{y}_i$ is predicted output.

to make $J(\theta_0, \theta_1)$ minimum or if we find minima of $J(\theta_0, \theta_1)$, then

$\frac{\partial J}{\partial \theta_0}$ and $\frac{\partial J}{\partial \theta_1}$ will be zero.

$$\frac{\partial J(\theta_0, \theta_1)}{\partial \theta_0} = 0$$

$$\Rightarrow \frac{\partial}{\partial \theta_0} \sum_{i=1}^{n} (y_i - \theta_0 - \theta_1 x_i)^2 = 0$$

$$\Rightarrow -2 \sum_{i=1}^{n} (y_i - \theta_0 - \theta_1 x_i) = 0$$

$$\Rightarrow \sum_{i=1}^{n} y_i - \sum_{i=1}^{n} \theta_0 - \theta_1 \sum_{i=1}^{n} x_i = 0$$

$$\Rightarrow \bar{y} - n\theta_0 - \theta_1 \bar{x}$$

$$\Rightarrow \sum_{i=1}^{n} y_i - n\theta_0 - \theta_1 \sum_{i=1}^{n} x_i = 0$$

$$\Rightarrow \frac{1}{n} \sum_{i=1}^{n} y_i - \theta_0 - \theta_1 \cdot \frac{1}{n} \sum_{i=1}^{n} x_i = 0$$

$$\Rightarrow \bar{y} - \theta_0 - \theta_1 \bar{x} = 0$$

$$\Rightarrow \theta_0 = \bar{y} - \theta_1 \bar{x} \quad \ldots (1)$$

Now, $\frac{\partial J(\theta_0, \theta_1)}{\partial \theta_1} = 0$

$$\Rightarrow \frac{\partial}{\partial \theta_1} \sum_{i=1}^{n} (y_i - \theta_0 - \theta_1 x_i)^2 = 0$$

$$\Rightarrow \frac{\partial}{\partial \theta_1} \sum_{i=1}^{n} (y_i - \bar{y} + \theta_1 \bar{x} - \theta_1 x_i)^2 = 0$$

$$\Rightarrow 2 \sum_{i=1}^{n} (y_i - \bar{y} + \theta_1 (\bar{x} - x_i))(\bar{x} - x_i) = 0$$

$$\Rightarrow \sum_{i=1}^{n} ((y_i - \bar{y}) + \theta_1 (\bar{x} - x_i))(\bar{x} - x_i) = 0$$

$$\Rightarrow \sum_{i=1}^{n} \left((y_i - \bar{y})(\bar{x} - x_i) + \theta_1 (\bar{x} - x_i)^2 \right) = 0$$

$$\Rightarrow -\sum_{i=1}^{n} (y_i - \bar{y})(x_i - \bar{x}) + \sum_{i=1}^{n} \theta_1 (x_i - \bar{x})^2 = 0$$

$$\Rightarrow \theta_1 \sum_{i=1}^{n} (x_i - \bar{x})^2 = \sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y})$$

$$\Rightarrow \theta_1 = \frac{\sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n} (x_i - \bar{x})^2} \quad \text{...(i)}$$

also can be write, $\theta_1 = \frac{cov(x, y)}{var(x)}$

put value of θ_1 in eq(1), so we can find θ_0 also.

Multiple Linear Regression (OLS)

$$\hat{y} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \hat{y}_3 \\ \vdots \\ \hat{y}_n \end{bmatrix} = \begin{bmatrix} \beta_0 & \beta_1 x_{11} & \beta_2 x_{12} & \cdots & \beta_m x_{1m} \\ \beta_0 & \beta_1 x_{21} & \beta_2 x_{22} & \cdots & \beta_m x_{2m} \\ \beta_0 & \beta_1 x_{31} & \beta_2 x_{32} & \cdots & \beta_m x_{3m} \\ \vdots & & & & \\ \beta_0 & \beta_1 x_{n1} & \beta_2 x_{n2} & \cdots & \beta_m x_{nm} \end{bmatrix}$$

$$\begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \hat{y}_3 \\ \vdots \\ \hat{y}_n \end{bmatrix} = \begin{bmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1m} \\ 1 & x_{21} & x_{22} & \cdots & x_{2m} \\ 1 & x_{31} & x_{32} & \cdots & x_{3m} \\ \vdots & & & & \\ 1 & x_{n1} & x_{n2} & \cdots & x_{nm} \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \\ \vdots \\ \beta_m \end{bmatrix}$$

$$\therefore \hat{y} = X\beta$$

errors, $e = y - \hat{y}$, y is actual
$\hat{y}$ is predicted
e also referred as ε

$$y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} \quad e = \begin{bmatrix} y_1 - \hat{y}_1 \\ y_2 - \hat{y}_2 \\ \vdots \\ y_n - \hat{y}_n \end{bmatrix}$$

Error function / cost function,

$$E = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

$$\therefore E = e^T \cdot e$$

$$\therefore E = (y - \hat{y})^T (y - \hat{y})$$

$$= (y^T - \hat{y}^T)(y - \hat{y})$$

$$= y^T y - y$$

$$= (y^T - (X\beta)^T)(y - (X\beta))$$

$$= y^T y - y^T (X\beta) - (X\beta)^T y + (X\beta)^T (X\beta)$$

Now, $y^T (X\beta) = (X\beta)^T y$ because both are 1×1 matrix

$$-2\beta^T X^T y$$

$$\therefore E = y^T y - 2y^T X\beta + \beta^T X^T X\beta$$

Now, as we have to minimize the cost function E,

$$\frac{dE}{d\beta} = 0$$

~~$-2Y^TX + 2X^TX\beta = 0$~~

~~$\Rightarrow X^TX\beta = X$~~

$$-2X^TY + 2X^TX\beta = 0$$

$$\Rightarrow X^TX\beta = X^TY$$

$$\Rightarrow \beta = (X^TX)^{-1}X^TY$$ ✓

It is very time consuming as it involves inverse of matrix $(X^TX)^{-1}$.

For any Linear Regression,

$$\hat{Y} = X\beta$$

$$Y = \hat{Y} + \varepsilon = X\beta + \varepsilon$$

where, $\hat{Y}$ = Predicted

Y = Actual

$\varepsilon = Y - \hat{Y}$

= difference between actual and predicted

ε also called as noise

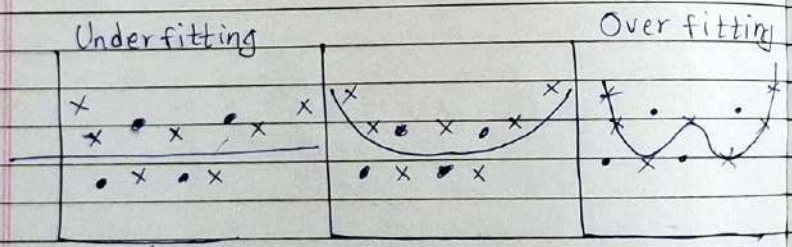
Bias and Variance

Bias : The inability of ML model to truly capture the relationship in the training data.

Variance : Variance indicates how much the estimate of the target function will differ ~~if~~ from ~~its~~ actual random variable, if different data used.

Generally, Models with high bias will have low variance. Models with high variance will have a low bias.

High Bias → Under fitting
High Variance → Overfitting



High bias Low variance | Low variance | Low bias High variance

Example :

for Cat classification,

Train set error (Training)	Test ~~error~~ set error (Testing or Validation)	Conclusion
1 %	3 %	low bias, low variance (best model)
2 %	11 %	low bias, high variance (Overfitting)
15 %	16 %	high bias, low varincace (Underfitting)
12 %	25 %	high bias, high variance

MAE, MSE, RMSE, R-SQUARE, ADJUSTED R-SQUARE

MAE : (Mean Absolute Error)

~~$MAE = \frac{1}{n}\sum|(y_i - \hat{y})|$~~

$\hat{y}$ = predicted

y = actual

$$MAE = E|Y - \hat{Y}|$$

MSE : (Mean squared Error)

$$MSE = E(Y - \hat{Y})^2$$

RMSE : (Root Mean Squared Error)

$$RMSE = \sqrt{MSE}$$

R^2 (coefficient of determination) :

measures how well a statistical model predicts an outcome. range is [0, 1]

$$R^2 = 1 - \frac{E(Y - \hat{Y})^2}{E(Y - \bar{Y})^2}$$

where, Y = actual

$\hat{Y}$ = Predicted

$\bar{Y}$ = mean of Y (constant)

Adjusted R^2 :

- See krish naik's ~~vc~~ video about this.

$$\text{Adjusted } R^2 = 1 - \frac{(1 - R^2)(N - 1)}{(N - P - 1)}$$

As, P increases and R^2 not increases much then Adjusted R^2 will decreases

N = no. of datapoints

P = No. of independent features

- it is basically used for prevent unnecessary features.

Gradient Descent

→ Batch Gradient Descent :

$$\hat{Y} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \hat{y}_3 \\ \vdots \\ \hat{y}_n \end{bmatrix} = \begin{bmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1m} \\ 1 & x_{21} & x_{22} & \cdots & x_{2m} \\ 1 & x_{31} & x_{32} & \cdots & x_{3m} \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_{n1} & x_{n2} & \cdots & x_{nm} \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \\ \vdots \\ \beta_m \end{bmatrix}$$

$\therefore \hat{Y} = X\beta$

$\hat{y}$ = Predicted

y = Actual

n = no. of datapoints

m = no. of independent features - I

Loss function,

$$L = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 = E(y - \hat{y})$$

for finding minimum of L we have to make their slopes with $\beta_0, \beta_1, \beta_2 \ldots$ to be zero.

So, Let's find all the slope.

$$\frac{\partial L}{\partial \beta_0} = \frac{-2}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)$$

$$L = \frac{1}{n}\sum_{i=1}^{n}(y_i - (\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \ldots + \beta_m x_{im}))^2$$

$$\frac{\partial L}{\partial \beta_1} = \frac{-2}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)x_{i1}$$

$$\frac{\partial L}{\partial \beta_2} = \frac{-2}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)x_{i2}$$

$$\frac{\partial L}{\partial \beta_m} = \frac{-2}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)x_{im}$$

Now, by setting Learning rate and epochs do looping and make all these slope nearby zero, so that we can find intercept and coefficients (β) for our model.

Stochastic Gradient Descent :

In this GD we have to change parameters by visiting every row (by randomly).

Advantage of this GD is that, we don't have to ~~do~~ process on entire dataset at a time rather we can go one by one row and updat the parameters

Mini Batch Gradient Descent :

In this GD we have to split dataset into n Batches (random) so that we can go by all these batches and do updation on parameters.

It is in between Batch GD and stochastic GD. And In this GD also we don't have to process on entire dataset at a time.

Polynomial Linear Regression :

- Polynomial LR is derived from Multiple LR.
- Let's say we have X and Y as input and output.

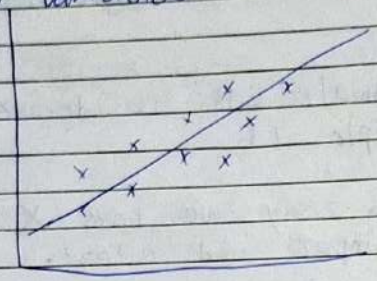
we have to find Polynomial function of X such that it can predict Y. for that we will take three columns X^0, X^1, X^2 as input. and we are assuming that these are independent columns. Now apply Multiple LR such that

$$\hat{y} = \beta_0 + \beta_1 X^0 + \beta_2 X^1 + \beta_3 X^2$$

find $\beta_0, \beta_1, \beta_2, \beta_3$.

Ridge Regression (L2 norm)

for two variables :



$$L = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 + \lambda m^2$$

λ = alpha (constant)

$$\hat{y}_i = mx_i + b$$

~~taking~~, $\frac{\partial L}{\partial m} = 0$

by taking $\frac{\partial L}{\partial b} = 0$

we get $b = \bar{y} - m\bar{x}$

by taking $\frac{\partial L}{\partial m} = 0$

$$\frac{\partial}{\partial m}\left(\sum_{i=1}^{n} (y_i - (mx_i + \bar{y} - m\bar{x}))^2 + \lambda m^2\right) = 0$$

$$\Rightarrow \sum_{i=1}^{n} 2(y_i - mx_i - \bar{y} + m\bar{x})(-x_i + \bar{x}) + 2\lambda m = 0$$

$$\Rightarrow -2\sum_{i=1}^{n} (y_i - \bar{y} - m(x_i - \bar{x})(x_i - \bar{x}) + 2\lambda m = 0$$

$$\Rightarrow -\sum_{i=1}^{n} (y_i - \bar{y})(x_i - \bar{x}) + m\sum_{i=1}^{n} (x_i - \bar{x})^2 + \lambda m = 0$$

$$\Rightarrow m\left(\sum_{i=1}^{n} (x_i - \bar{x})^2 + \lambda\right) = \sum_{i=1}^{n} (y_i - \bar{y})(x_i - \bar{x})$$

$$\Rightarrow m = \frac{\sum_{i=1}^{n} (y_i - \bar{y})(x_i - \bar{x})}{\sum_{i=1}^{n} (x_i - \bar{x})^2 + \lambda}$$

Ridge regression for multiple variables,

$$L = (y-\hat{y})^T(y-\hat{y}) + \lambda \|w\|^2 \quad (\beta)$$

$$\therefore L = (\hat{y}-y)^T(\hat{y}-y) + \lambda w^T w$$

$$= (xw-y)^T(xw-y) + \lambda w^T w$$

$$= (w^T x^T - y^T)(xw-y) + \lambda w^T w$$

$$\therefore L = w^T x^T x w - w^T x^T y - y^T x w + y^T y + \lambda w^T w$$

$$\therefore L = w^T x^T x w - 2 w^T x^T y + y^T y + \lambda w^T w$$

$$\frac{\partial L}{\partial w} = 0$$

$$\Rightarrow 2x^T x w - 2x^T y + 2\lambda w = 0$$

$$\Rightarrow (x^T x + \lambda I) w = x^T y$$

$$\Rightarrow w = (x^T x + \lambda I)^{-1} x^T y \quad \therefore \checkmark$$

Ridge regression using Gradient Descent:

$$L = (y-\hat{y})^T(y-\hat{y}) + \lambda w^T w$$

$$L = w^T x^T x w - 2 w^T x^T y + y^T y + \lambda w^T w$$

$$\frac{\partial L}{\partial w} = \frac{1}{2}\left[2 x^T x w - 2 x^T y + 2w\right]$$

$$\frac{\partial L}{\partial w} = x^T x w - x^T y + w \quad \left\{\text{Just ignored } \frac{2}{2}\right\}$$

for epochs time

$$w = w - lr^* \frac{\partial L}{\partial w}$$

Lasso Regression (L1 norm)

~~L = (y - y~~ $b = \bar{y} - m\bar{x}$

$$L = \sum_{i=1}^{n} (y_i - (mx_i - b))^2 + 2\lambda|m|$$

$$\therefore L = \sum_{i=1}^{n} (y_i - mx_i - \bar{y} + m\bar{x})^2 + 2\lambda|m|$$

$$\frac{dL}{dm} = 0$$

$$\Rightarrow 2\sum_{i=1}^{n} (y_i - mx_i - \bar{y} - m\bar{x})(-x_i + \bar{x}) + 2\lambda\frac{|m|}{m} = 0$$

$$\Rightarrow -\sum_{i=1}^{n} ((y_i - \bar{y}) - m(x_i - \bar{x}))(x_i - \bar{x}) + \lambda\frac{|m|}{m} = 0$$

$$\Rightarrow m = \frac{\sum_{i=1}^{n} (y_i - \bar{y})(x_i - \bar{x}) - \lambda\left(\frac{|m|}{m}\right)}{\sum_{i=1}^{n} (x_i - \bar{x})^2}$$

$m < 0$, $m = \dfrac{\sum_{i=1}^{n} (y_i - \bar{y})(x_i - \bar{x}) + \lambda}{\sum_{i=1}^{n} (x_i - \bar{x})^2}$

$m > 0$, $m = \dfrac{\sum_{i=1}^{n} (y_i - \bar{y})(x_i - \bar{x}) - \lambda}{\sum_{i=1}^{n} (x_i - \bar{x})^2}$

for multiple variables,

$$W = (X^T X)^{-1}(X^T y \mp \lambda I), \quad |w| > 0$$

$$W = (X^T X)^{-1}(X^T y + \lambda I), \quad |w| < 0$$

- In Lasso regression after some higher value of alpha we will get zero (absolute) as a coefficient. (alpha in numerator)

whereas in Ridge regression there is coefficients are nearly zero, but not absolute zero. (alpha in denomenator)

- So lasso regression maybe doing feature selection.

- So, ~~if~~ you can use ridge when you know that all of your columns are important.

and you can use lasso when you know that some columns are not important, so by applying lasso you can make their coefficients zero.

Elastic Net Regression :

$$L = (y - \hat{y})^2 + a\|w\|^2 + b\|w\|$$

alpha (λ) = a + b

l1-ratio = $\frac{a}{a+b}$

Logistic Regression

Perceptron Trick

- This is based on randomness and you will not get best fit line. but you can make descent model out of this.

- first we have to initialize weights and bias

then based on random point we have to manipulate weights and bias.

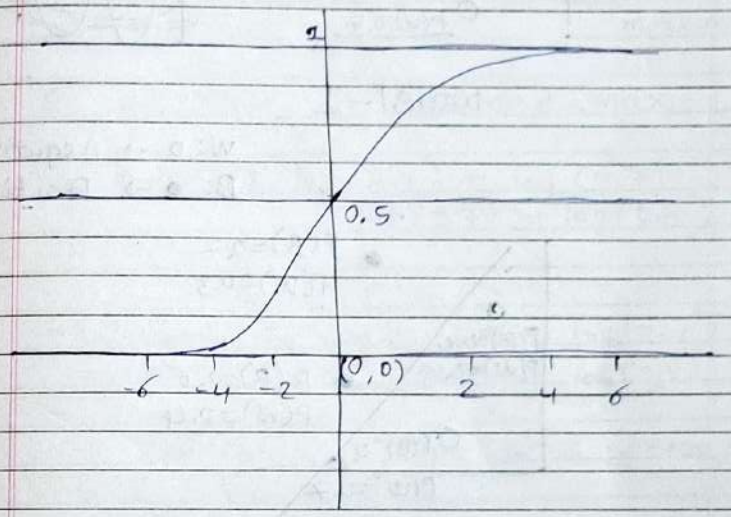
if point belongs to Positive part but function says Negative part
then do $W = W + X$ [that point's index]

and in the opposite case you have to do $W = W - X$ [that point's index]

- The main Problem with perceptron trick is that you will not get the best fit line. you may get rough line, which is classifying the two classes but not in very good way.

- Sigmoid function :

$$f(x) = \frac{1}{1 + e^{-x}}$$

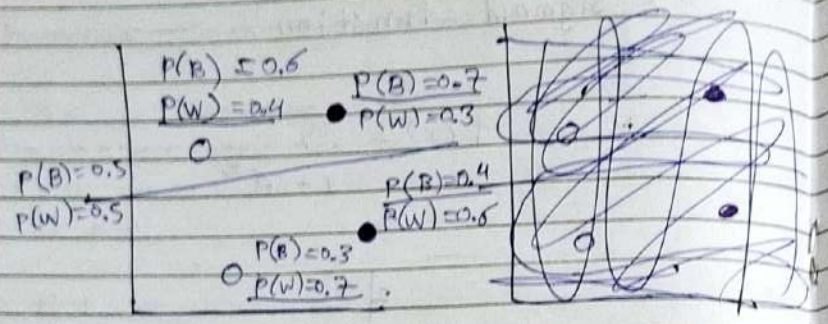


- We can use sigmoid function to calculate $\hat{Y}$, and also it is ~~of~~ optimized than normal (0 ro 1).

and this value of sigmoid we can take as the probability (rough).

$\hat{Y} = \sigma(z)$ ← sigma function

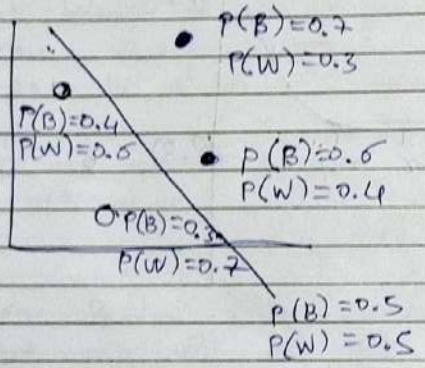
↳ $\sum w_i x_i$



model - I

W: ○ → Negative

B: ● → Positive



model - 2

for model - I,

~~max = 0.7 × 0.4 × 0.~~

max = 0.4 × 0.7 × 0.7 × 0.4 = 0.

↑ best model will have this term maximum.

So, we have to maximize max

$\log(max) = \log(0.4) + \log(0.7) + \log(0.7) + \log(0.4)$

$-\log(max) = -\log(0.4) - \log(0.7) - \log(0.7) - \log(0.4)$

↑ Loss function (Binary Cross entropy)

we have to minimize Loss function.

General,

$$L = \frac{-1}{n} \sum_{i=1}^{n} Y_i \log(\hat{Y}_i) + (1-Y_i)\log(1-\hat{Y}_i)$$

also, $L = \frac{-1}{n}\left[Y \log(\sigma(XW)) + (1-Y)\log(1-\sigma(XW)) \right]$

$$\therefore \frac{dL}{dw} = -\frac{1}{n}\left[Y \frac{d}{dw} \log(\sigma(xw)) + (1-Y)\frac{d}{dw}\log(1-\sigma(xw)) \right]$$

$$= -\frac{1}{n}\left[y \cdot \frac{1}{\sigma(xw)} \frac{d}{dw}\sigma(xw) + (1-y)\cdot \frac{1}{1-\sigma(xw)} \frac{d}{dw}(1-\sigma(xw)) \right]$$

$$= -\frac{1}{n}\left[y \cdot \frac{1}{\hat{y}} \sigma(xw)\cdot(1-\sigma(xw))\cdot x + \frac{(1-y)}{1-\hat{y}} \cdot(-\sigma(xw)(1-\sigma(xw))\cdot x) \right]$$

$$= -\frac{1}{n}\left[y(1-\hat{y})x - \hat{y}(1-y)x \right]$$

$$= -\frac{1}{n}\left[y(1-\hat{y}) - \hat{y}(1-y) \right] x$$

$$= -\frac{1}{n}\left[y - y\hat{y} - \hat{y} + \hat{y}y \right] x \quad (y\hat{y}: 1\times1,\ \hat{y}y: 1\times1)$$

$$\frac{dL}{dw} = -\frac{1}{n}\left[y - \hat{y} \right] x$$

Accuracy :

Accuracy for any classificating model is equals to,

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{All Predictions}}$$

$$= \frac{\checkmark\ \checkmark\ \checkmark\ \checkmark}{\checkmark\ \times\ \checkmark\ \checkmark\ \times\ \times\ \checkmark}$$

Confusion Matrix :

It presents a table of all the predicted and actual values of a classifier.

Actual \ Predicted	1	0	
1	True Positive	False Negative	Type II error
0	False Positive	True Negative	

Type I error

Accuracy may mislead you some times so you can go with confusion matrix at that time.

Precision : What proportion of predicted positives is truly positive?

$$\text{precision} = \frac{n(TP)}{n(TP) + n(FP)}$$

~~True positives out of predicted True positives~~

Recall :

$$\text{recall} = \frac{n(TP)}{n(TP) + n(FN)}$$

- Analyze Precision and Recall in the confusion matrix.

- Use Precision and Recall as per which error (type I or type II) are dangerous.

F1 Score :

$$\text{F1 score} = \frac{2PR}{P+R}$$

P = Precision
R = Recall

F1 score is inside of lowe value. (analyze)

Conclusion :

		Predicted: Dog	Cat	Rabbit	total (weights)
Actual	Dog	25	5	10	40
	Cat	0	30	4	34
	Rabbit	4	10	20	34
	total	29	45	34	

3×3

$$P_D = \frac{25}{29}, \quad P_C = \frac{30}{45}, \quad P_R = \frac{20}{34}$$

$$R_D = \frac{25}{40}, \quad R_C = \frac{30}{34}, \quad R_R = \frac{20}{34}$$

Macro precision

$$P_{macro} = \frac{P_D + P_C + P_B}{3}$$

Macro Recall

$$R_{macro} = \frac{R_D + R_C + R_B}{3}$$

Weighted Precision

$$W_D = \frac{40}{108}, \quad W_C = \frac{34}{108}, \quad W_B = \frac{34}{108}$$

$$P_{weighted} = W_D P_D + W_C P_C + W_B P_B$$

Weighted Recall

$$R_{weighted} = W_D R_D + W_C R_C + W_B R_B$$

Softmax Regression
(Multiple Logistic Regression) :

Softmax function,

$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{n} e^{z_j}}$$

~~$(\hat{y})_i = e$~~

$$(\hat{y})_i = \sigma(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{n} e^{z_j}}$$

$$z = X^T \beta$$

$$z_i = X_i \beta$$

$$\hat{y}_i = \underset{k}{\operatorname{argmax}}\ \sigma(X_i)_k$$

Cost function (Gross entropy / Cross function),

$$J(\theta) = \frac{-1}{m} \sum_{i=1}^{n} \sum_{k=1}^{K} y_k^{(i)} \log\left(\hat{y}_k^{(i)}\right)$$

- We can use polynomial features in logistic regression. for that we have to provide input columns as per the applying polynomial features.

if X_1, X_2 are inputs and y is output

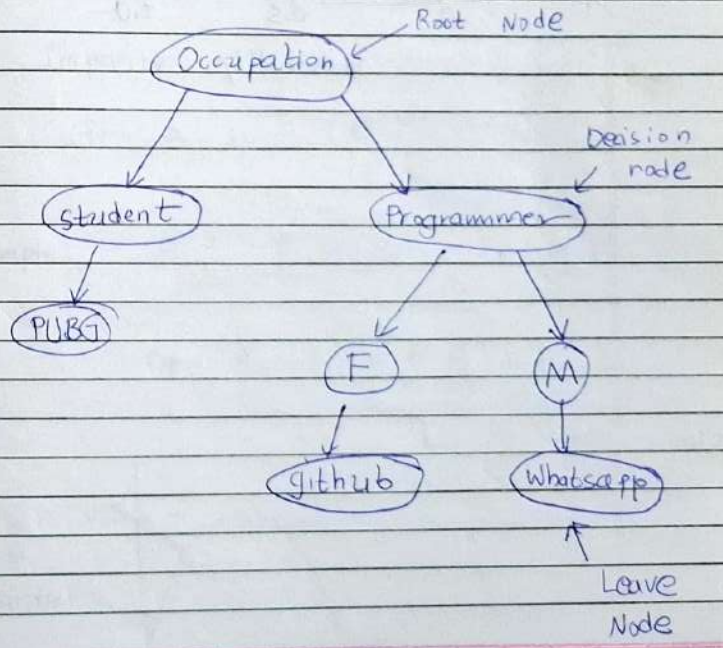
for degree 2 polynomial we can give inputs $X_1, X_1^2, X_2, X_2^2, X_1X_2$.

$$\therefore Y = \beta_1 X_1 + \beta_2 X_1^2 + \beta_3 X_2 + \beta_4 X_2^2 + \beta_5 X_1 X_2 + \beta_0$$

Decision Trees

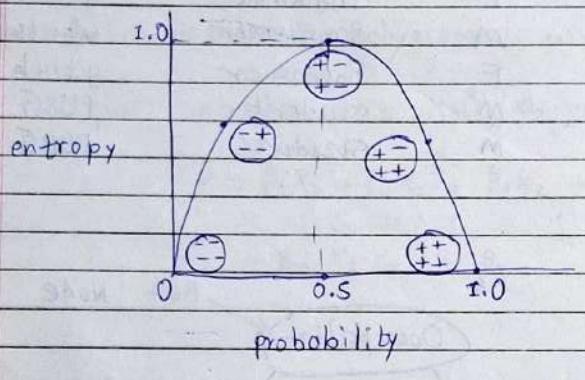
Decision Trees :

Gender	Occupation	Suggestion
F	Student	PUBG
F	Programmer	github
M	Programmer	Whatsapp
F	Programmer	github
M	student	PUBG
M	Student	PUBG



Entropy: Measure of impurity.

$$E(S) = \sum_{i=1}^{n} -P_i \log_2 P_i$$



Information Gain:

Information gain is a metric used to train Decision Trees. Constructing a decision tree is all about finding attribute that returns the highest information gain.

$$\text{Information Gain} = E(\text{Parent}) - E_{\substack{\text{weighted} \\ \text{average}}}(\text{childrens})$$

E = Entropy

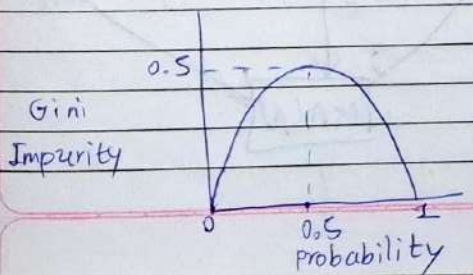
Gini Impurity

$$G_I = 1 - \left(\sum_{i=1}^{n} P_i^2\right)$$

example: $E = -P_1 \log_2 P_1 - P_2 \log_2 P_2$

$$G_I = 1 - (P_1^2 + P_2^2)$$

For 2 classes, 1 and 2

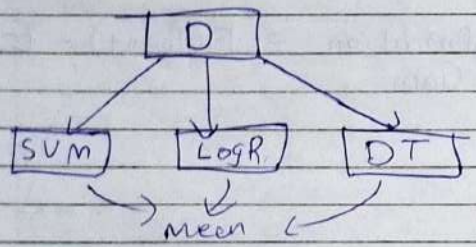


Ensemble Learning

Types :

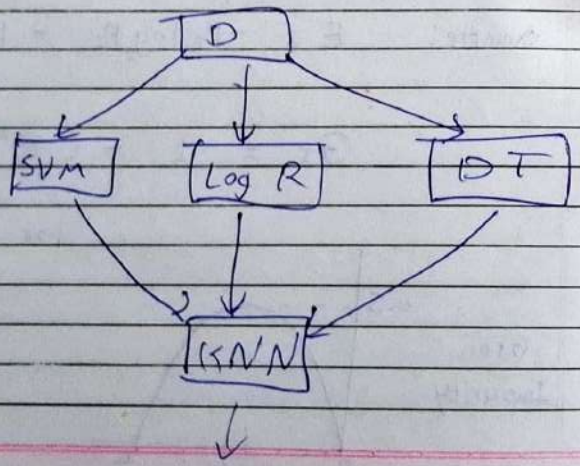
Voting ensemble:

- Models are different, Data is same



Stacking ensemble :

- Expand Voting ensemble with one final model



- In staking example:

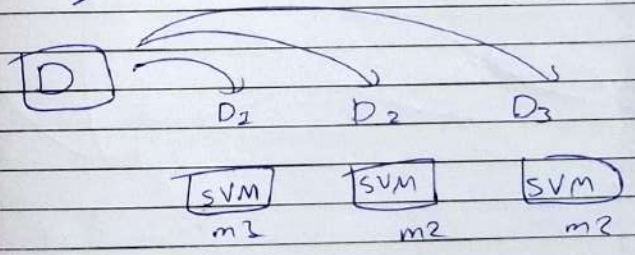
for SVM, log R, DT have training data as D.

but for KNN training data comes from SVM log R and DT Result.

Bagging Ensemble : (random forest (DTC))
(bootstrapped aggregation)

- Same ~~Mode~~ models, but different data

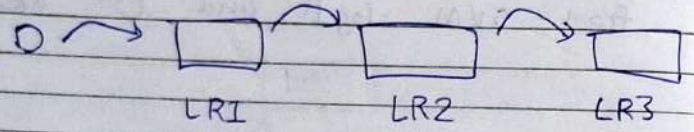
- data choosed by bootstrapping



D_1, D_2, D_3 are randomly selected from D for given size.

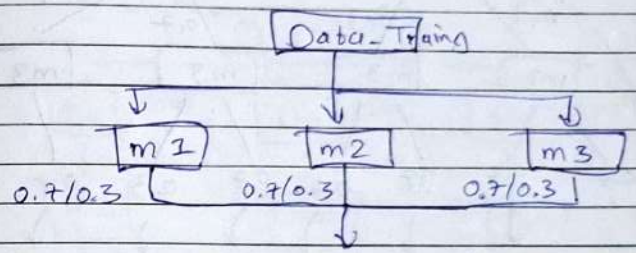
Boosting:

It boosts the model after every model training.

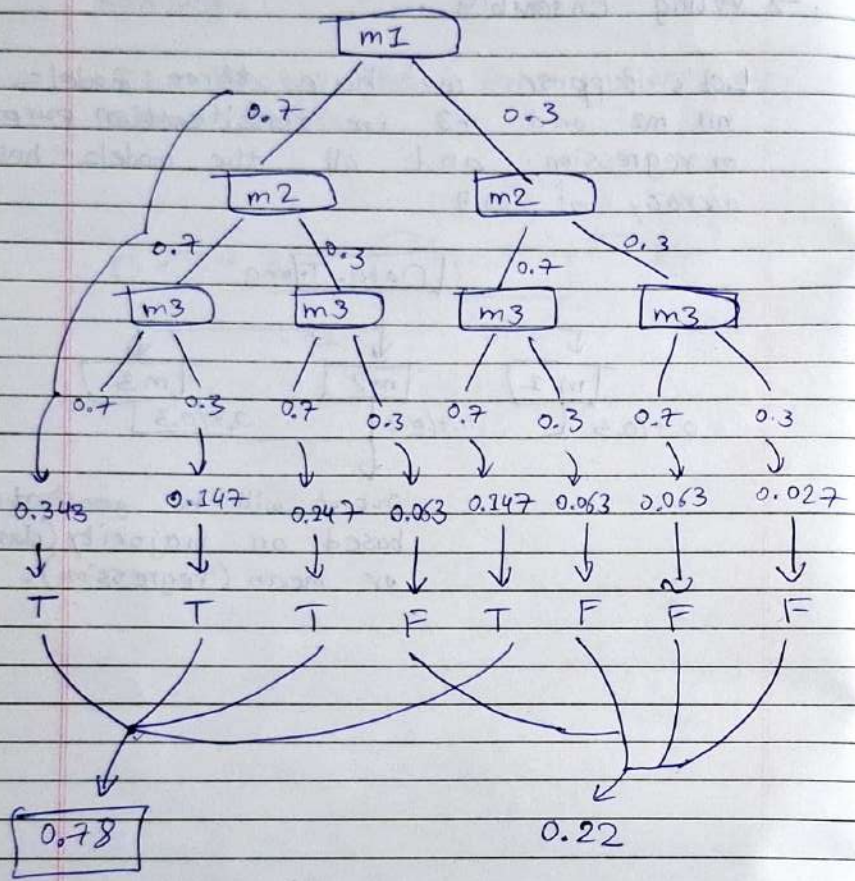


→ Voting Ensemble :

Let's suppose we have three models m1, m2 and m3 for classification purpose or regression. and all the models have accracy of 0.7



Output will be generated based on majority (classi. or mean (regression).

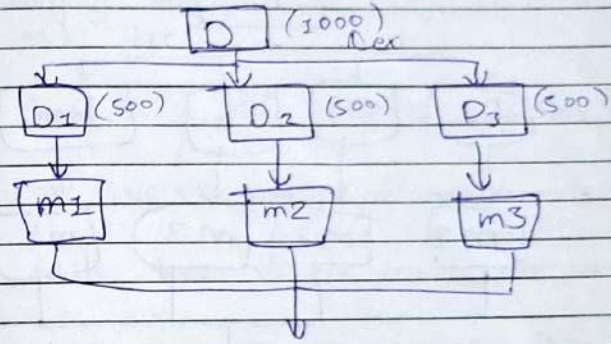


Accuracy Increased

Bagging :

(Bootstraping + Aggregation)

We have same models but dataset will be different (bootstraped)

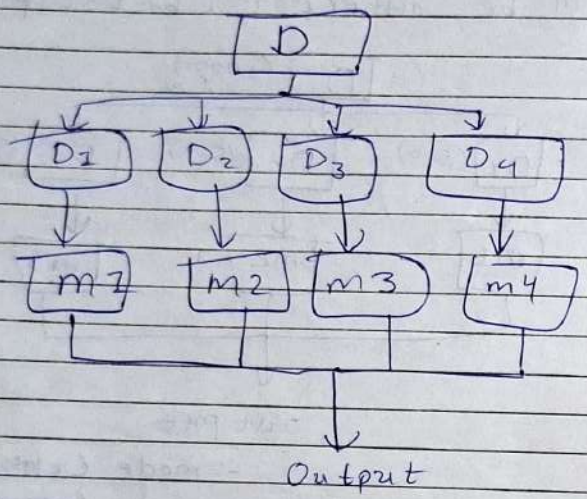


output

- mode (classification)
- mean (regression)

Random Forest :

(Bagging Ensemble with estimator as Decision Tree)



Here D_1, D_2, D_3, D_4 are sampled data from D.

and m_1, m_2, m_3, m_4 are Decision Tree models

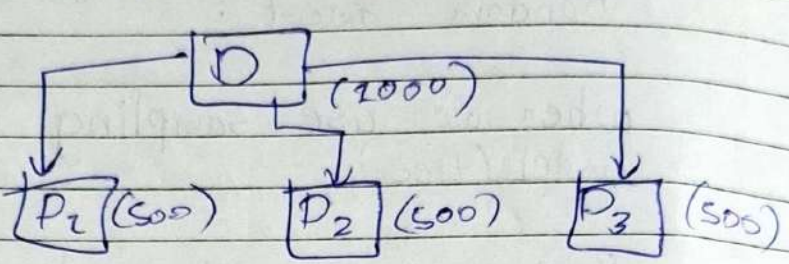
Difference between Bagging (Tree) and Random forest :

when we use sampling for each model (tree),

Bagging samples data only once for each tree.

whereas, Random forest samples data at every node of tree. so, for each tree random forest samples data for multiple times, But Bagging samples data only once for each tree.

OOB Score (out of Box Score):



When you make samples from data by bootstrapping there will be some samples who will not been used ~~for any~~ by any sample. So there will be some unused data. We can use these data for validation (testing). After validating models by these data whatever accuracy comes out called as <u>oob-score</u>.

So, oob score is accuracy score which is got by using unused training data.

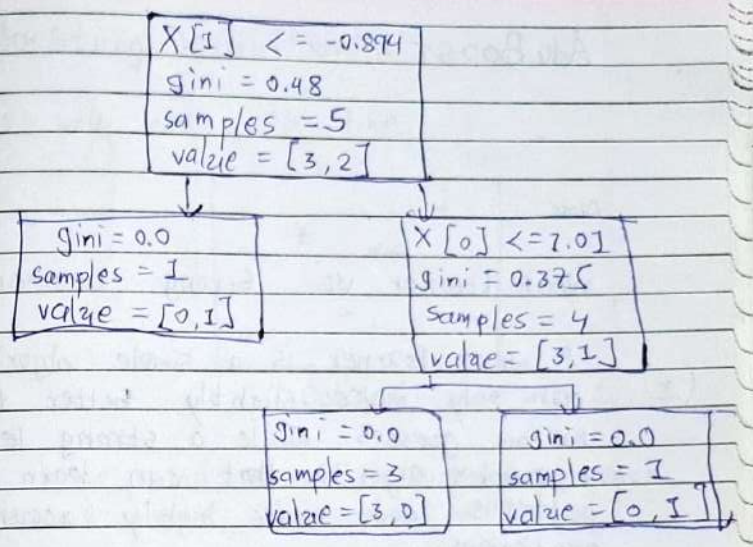
Feature importance using decision tree classifier : (gini importance) :

$$n_{imp} = \frac{N\_t}{N}\left[\text{impurity} - \left(\frac{N\_t\_r}{N\_t} \times \text{right impurity}\right) - \left(\frac{N\_t\_l}{N\_t} \times \text{left impurity}\right)\right]$$

n_{imp} : importance for each node

~~$f_{imp} = \sum n_{imp}$~~

$$(f_{imp})_k = \frac{\sum_{\text{Nodes split on feature } k} n_{imp}}{\sum_{\text{all nodes}} n_{imp}}$$



$$(n_{imp})_1 = \frac{5}{5}\left[0.48 - \left(\frac{1}{5} \times 0.0\right) - \left(\frac{4}{5} \times 0.375\right)\right]$$

$$= 0.\sim$$

$$(n_{imp})_0 = \frac{4}{5}\left[0.375 - \left(\frac{3}{4} \times 0.0\right) - \left(\frac{1}{4} \times 0.0\right)\right]$$

$$\approx 0.\sim$$

$$(f_{imp})_1 = \frac{(n_{imp})_1}{(n_{imp})_1 + (n_{imp})_2'} \qquad (f_{imp})_2 = \frac{(n_{imp})_2}{11}$$

AdaBoost :

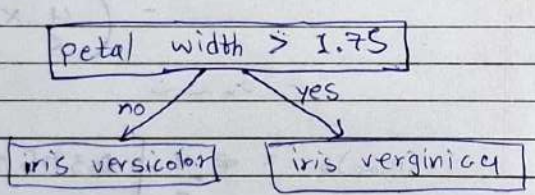
Note:

Weak Learner vs Strong Learner:

A weak learner is a simple algorithm that can only make slightly better than random guesses. while a strong learner is a complex algorith that can learn complex patterns and make highly accurate predictions.

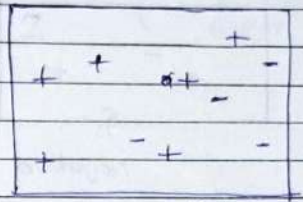
Decision stump: is a machine learning model consisting of a one-level decision tree. (max_depth = 1)

eg.



→ * Boosting Intuition :

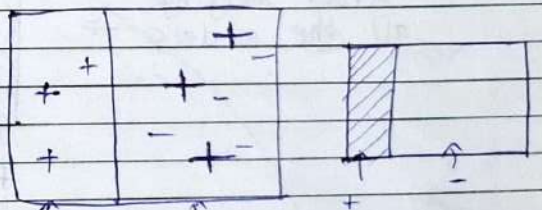
- let's say data is like



Data (classes, +1, -1)

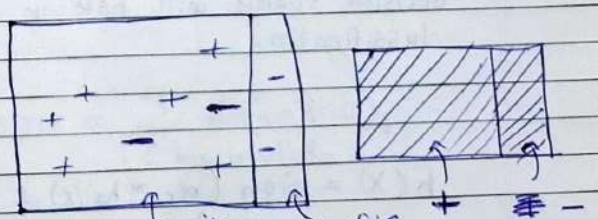
we have find best Decision stump each time we make new model.

m1 :



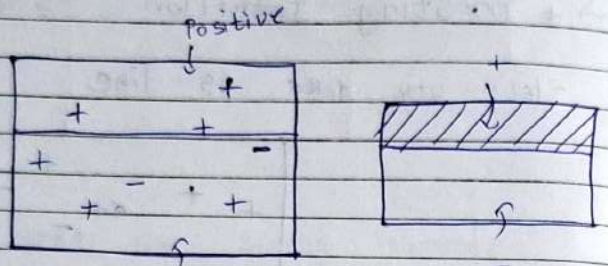
α_1

m2 :



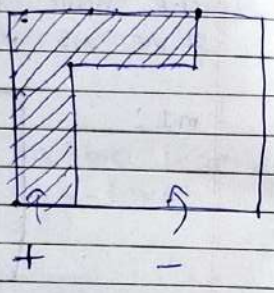
α_2

m3 :



α_3

After merging all the models →



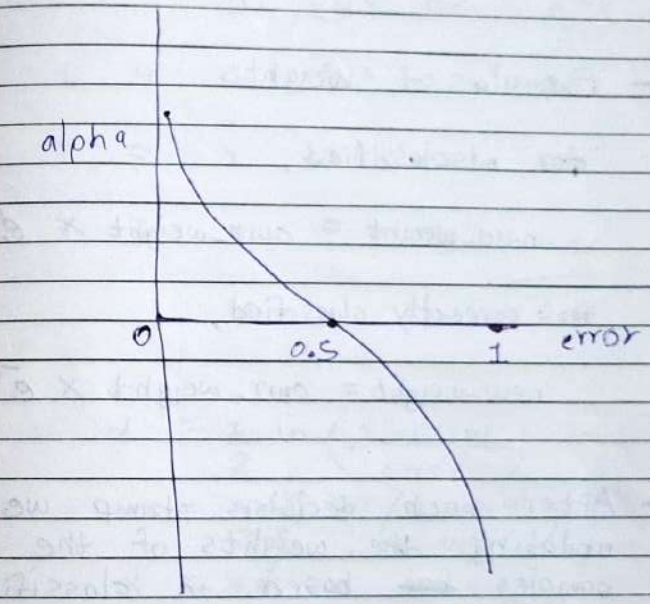
- Alpha is how much influence any decision stump will have in the final classification.

$$h(X) = \text{sign}\left(\underbrace{\alpha_1 * h_1(x) + \alpha_2 * h_2(x) + \alpha_3 * h_3(x)}\right)$$

if positive → +1
if Negative → -1

Formula for α :

$$\alpha = \frac{1}{2} \ln\left(\frac{1 - \text{error}}{\text{error}}\right)$$



$$\text{error} = \sum_{i \in \text{misclassified elements}} (\text{weight})_i$$

- Weights of all samples at beginning is same ($1/n$). but as we move forward we have to change the weights of samples in basis of misclassified or correctly classified.

- Formula of weights :

for misclassified,

$$new\_weight = curr\_weight \times e^{\alpha_{prev}}$$

for correctly classified,

$$new\_weight = curr\_weight \times e^{-\alpha_{prev}}$$

- After each decision stump we are updating the weights of the samples ~~bas~~ based on classified, misclassified and α_{prev} (alpha for that stump).

Example:

X_1	X_2	y	y_{pred}	Weight	Weight updated	normalizing weights
3	7	1	1	0.2	0.16	0.166
2	9	0	(1) x	0.2	0.24	0.25
3	4	1	(0) x	0.2	0.24	0.25
9	8	0	0	0.2	0.16	0.166
3	7	0	0	0.2	0.16	0.166
					0.96	

$$\alpha = \frac{1}{2} \ln\left(\frac{1 - error}{error}\right)$$

$$= \frac{1}{2} \ln\left(\frac{1 - 0.4}{0.4}\right)$$

$$\alpha = 0.20$$

find new weight for all samples based on alpha.

now, based on this updated weights we have to make new updated dataset for upcoming decision stump.

Index	X_1	X_2	Y	new-wt	range
0	3	7	1	0.166	0 - 0.166
1	2	9	0	0.25	0.166 - 0.416
2	1	4	1	0.25	0.416 - 0.666
3	9	8	0	0.166	0.666 - 0.832
4	3	7	0	0.166	0.832 - 1.0

now take n random numbers between 0 and 1,

ex. 0.13 → 1
0.43 → 3
0.62 → 3
0.50 → 3
0.8 → 4

So, this ~~ind~~ randomly sampled indexes will generate new data. After ~~generatipie~~ the new data weights will again be $1/n$ for all samples.

- In final stage:

we have n models, so $\alpha_1, \alpha_2, \alpha_3 \dots \alpha_n$. and functions for generating predictions, $h_1(x), h_2(x), \dots h_n(x)$

$\therefore P = \alpha_1 h_1(x) + \alpha_2 h_2(x) + \dots + \alpha_n h_n(x)$

Baggin Vs Boosting

1) Types of model used:

Bagging → (Low Bias High variance) LBHV
ex. fully gram DT

Boosting → (High bias low variance) HBLV
ex. shallow DT (Decision stump)

2) Sequential vs Parallel

Baggin → Parallel learning

Boosting → sequential learning

3) Weightage of base learners:

Bagging → all base models have same weight

Boosting → all models have different weightages.

K - Means Clustering:

1) Decide no. of clusters (K)
- we have to decide value of K.

2) Initialize centroids
- initialize k centroids (randomly) for k clusters.

3) Assign clusters
- based on centroids and euclidean distance from each point decide clusters

~~4) Move centroids~~
~~for each centroid.~~

4) Move centroids:

- based on clusters and corresponding centroid define new centroid for each clusters.

5) When centroid being constant, that means we have got best clusters, so finish the process

- Now, for finding number of clusters (k) we have to analyz one graph (Elbow Method).

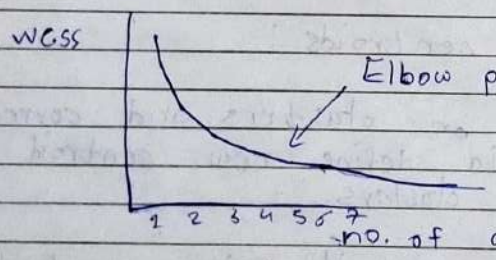
Note :

WCSS (Within-Cluster Sum of Square):

WCSS we are calculating for Elbow Method.

Elbow method is used to find the value of k.

WCSS is the sum of squared distance between each point and the centroid in a cluster.



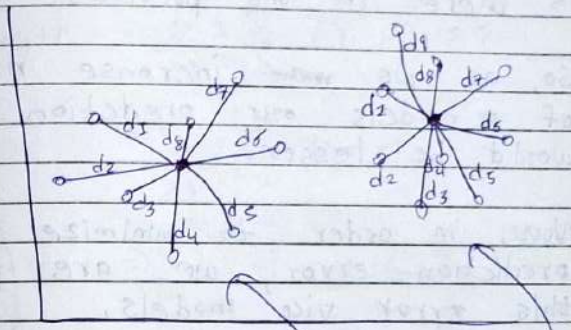
- We can find no. of clusters k from Elbow point.

- Elbow point is when graph becomes consistent

Let, $K_1 = 1$, $K_2 = 2$, $K_3 = 3$,

then, $WCSS_{K1} > WCSS_{K2} > WCSS_{K3}$

as we increase the value of k WCSS get decrement. when k equals to no. of data points WCSS become 0.



$$(WCSS)_{K=2} = (WCSS)_1 + (WCSS)_2$$

$$= \sum_{i=1}^{8} di^2 + \sum_{i=1}^{9} di^2$$

Gradient Boosting :

→ In gradient boosting first model is mean of the all training outcomes. → (loss function = Mean squared error)

→ now as we move forward, we have to minimize prediction error, which is there in the previous model.

→ So, as we ~~not~~ increase number of ~~s~~ models our predction error would be lesser.

→ Now, in order to minimize this prediction error, we are predicting this error vice models.

→ So, our first model is nothig bzt a mean
thengnwards, ~~a~~ models are predicting error which are there in the previous model.

→ Gradient boosting uses Additive modeling.

→ Gradient Boosting for Regression :

- Let's say we have $\{(x_i, y_i)\}_{i=1}^{n}$ training set.
Loss function $L(y, F(x))$,
number of iterations M.

I. Initialize $f_0(x) = \text{argmin}_\gamma \sum_{i=1}^{N} L(y_i, \gamma)$

Note : $\frac{\partial}{\partial \gamma} \frac{1}{2} \sum_{i=1}^{N} L(y_i, \gamma) = 0$

$\Rightarrow \frac{\partial}{\partial \gamma} \frac{1}{2} \sum_{i=1}^{N} (y_i - \gamma)^2 = 0$

$\Rightarrow \sum_{i=1}^{N} \frac{2}{2}(y_i - \gamma) \cdot (-1) = 0$.

$\Rightarrow \sum_{i=1}^{N} y_i - \sum_{i=1}^{N} \gamma = 0$

$\Rightarrow \sum_{i=1}^{N} y_i = N\gamma$

$\Rightarrow \gamma = \frac{1}{N} \sum_{i=1}^{N} y_i = \text{mean}(Y)$

$\therefore f_0(x) = \text{mean}(Y)$

2. for $m = 1$ to M :

(a) for $i = 1$ to N

$$r_{im} = -\left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)}\right]_{f=f_{m-1}}$$

Note :

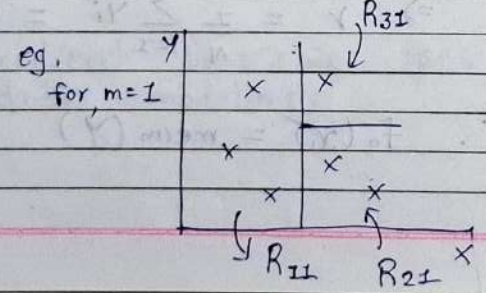
$$r_{im} = -\left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)}\right]_{f=f_{m-1}}$$

$$= -\left[\frac{\partial \frac{1}{2}(y_i - f(x_i))^2}{\partial f(x_i)}\right]_{f=f_{m-1}}$$

$$= \frac{-1}{2}\left[2(y_i - f(x_i))(-1)\right]_{f=f_{m-1}}$$

$$\therefore r_{im} = [y_i - f(x_i)]_{f=f_{m-1}}$$

(b) Fit a regression tree to the targets r_{im} giving terminal regions R_{jm}, $j = 1, 2, \ldots, J_m$



(c) for $j = 1, 2, \ldots J_m$

$$\gamma_{jm} = \arg\min_{\gamma} \sum_{x_i \in R_{jm}} L(y_i, f_{m-1}(x_i) + \gamma)$$

Note :

$$\gamma_{jm} = \arg\min_{\gamma} \sum_{x_i \in R_{jm}} L(y_i, f_{m-1}(x_i) + \gamma)$$

$$\therefore \frac{\partial}{\partial \gamma} \sum_{x_i \in R_{jm}} L(y_i, f_{m-1}(x_i) + \gamma) = 0$$

$$\Rightarrow \frac{\partial}{\partial \gamma} \sum_{x_i \in R_{jm}} \frac{1}{2}(y_i - (f_{m-1}(x_i) + \gamma))^2 = 0$$

$$\Rightarrow \sum_{x_i \in R_{jm}} (y_i - (f_{m-1}(x_i) + \gamma))(-1) = 0$$

$$\Rightarrow \sum_{x_i \in R_{jm}} (y_i - f_{m-1}(x_i)) - \sum_{x_i \in R_{jm}} \gamma = 0$$

$$\Rightarrow \sum_{x_i \in R_{jm}} (y_i - f_{m-1}(x_i)) = J_m \cdot \gamma$$

$$\Rightarrow \gamma = \frac{1}{J_m} \sum_{x_i \in R_{jm}} (y_i - f_{m-1}(x_i))$$

(d) update $f_m(x) = f_{m-1}(x) + (OT)_m$

3. Output $F(x) = f_0(x) + f_1(x) + \ldots f_M(x)$

- Gradient Boosting for classification:

Note:

$$\log(\text{odds}) = \log\left(\frac{n1}{n0}\right)$$

(n1: no. of 1, n0: no. of 0)

$$\text{probability for 1} = \frac{1}{1+e^{-\log(\text{odds})}}$$

Gradient Boosting (for classification) also works similar to as it works for regression.

- initial model (leaf) is log(odds)
- as we move forward we have make trees (leaf nodes 8 to 32) for predicting probability pseudo residual.

Note: to convert probability residual tree to log(odds) equation is,

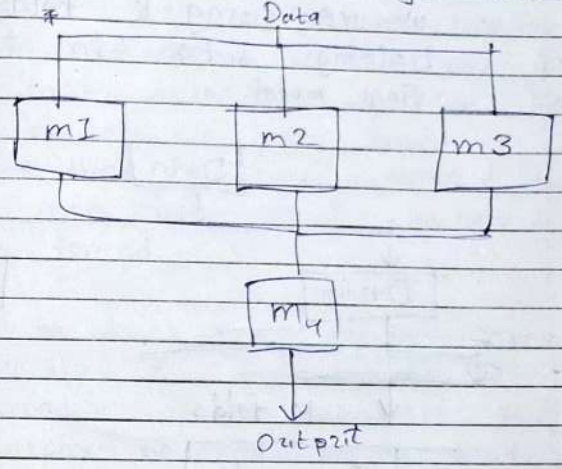
$$\log(\text{odds}) = \frac{\sum \text{Residual}}{\sum (\text{Previous Prob}^* (1 - \text{Pre. Prob}))}$$

- So, in predicting final output we are using log(odds). then final answer we will convert it into probability.

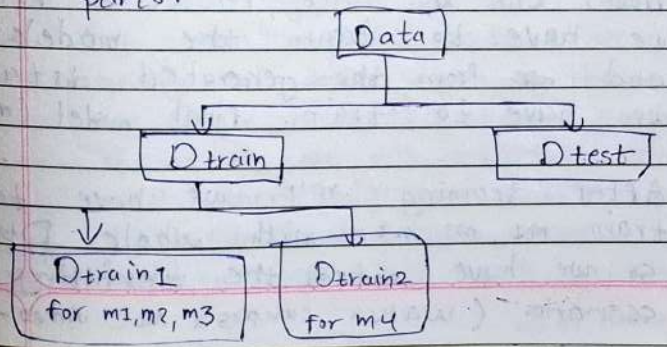
Staking and Blending Ensembles :

Blending :

- It is expansion of Voting Ensemble.

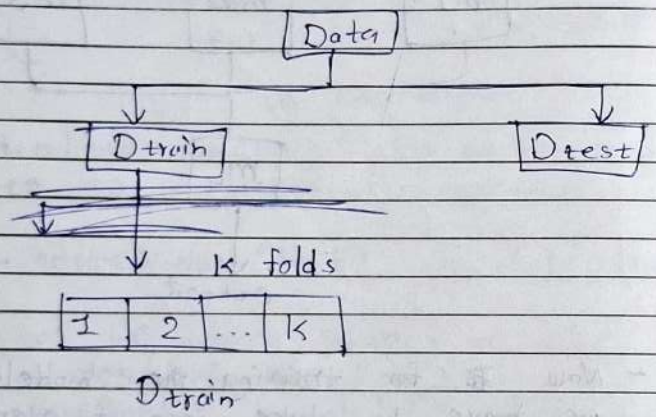


- Now for training the models, we have to take care of overfitting. And for that we have to split the training data into another two parts.



Stacking:

Now, in stacking concept is same but we are training the models we are using K folds of the training data for training the final model.



- Now, one by one, for each folds we have to train the models. and from the generated data we have to train final model (m4).

- After training m4, we have to train m1, m2, m3 with whole Dtrain. So, we have reduced the overfitting scenario. (watch campus x ML video - 109)

Agglomerative Hierarchical Clustering

- In this algorithm we are using bottom-up aproach.

- first we will assign all datapoints to an individual cluster. and then for each iteration we are merging this clusters in pair based on their similarity (distance) until one cluster or K clusters are formed.

- This whole process will create heirarchy of clusters. by observation and analysis of this heirarchy we can decide that how many clusters we have to make.

Divisive Hierarchical clustering

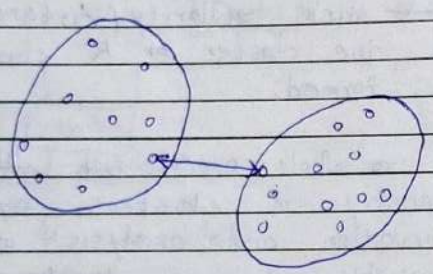
- In this algorithm we are using top-down aproach. first we will assign all data members to a single cluster. after each iteration we will separate clusters into other clusters based upon dissimilarity and this process repeats until we are left with n clusters.

n ↑ no. of data points

- for similarity or dissimilarity we are finding distance and this distance ~~are find~~ can be find ~~not~~ in many ways some of the ways are given below.

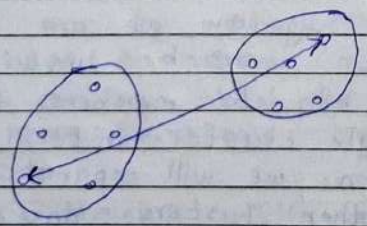
(i) single linkage :

- minimum distance.



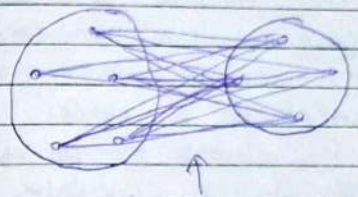
(ii) Complete linkage :

- maximum distance.



(iii) Average linkage

- Average distance of all the distances between all datapoints of both the clusters.



average of all this distances

K Nearest Neighbors :

- KNN is a type of supervised learing algorith used for both regression and classification. KNN triest to predict the correct class for the test data by calculating the distance between the test data and all the training points.
Then select the K number of points which is closest to the test data.
The KNN algorithm calculates the probability of the test data belonging to the classes of 'K' training data and class holds the highest probability will be selected. In the case of regression, the value of is the mean of the 'K' selected training points.

- Here, we are using Euclidean distance.

- Imp : As we increase the value of K, model will create underfitting.

If we decrease the value of K, model will create overfitting.

so, we have to find intermediate value of K.

- for very large datasets KNN is not applicable because it is very time consuming.

Assumptions of Linear Regression :

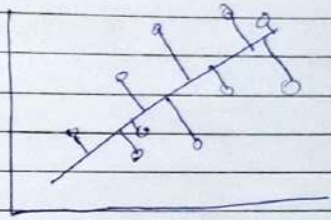
1. Linear Relationship

2. Multicollinearity :

Their should not be collinearity between any features.

All features should be independent.

3. Normality of Residual :

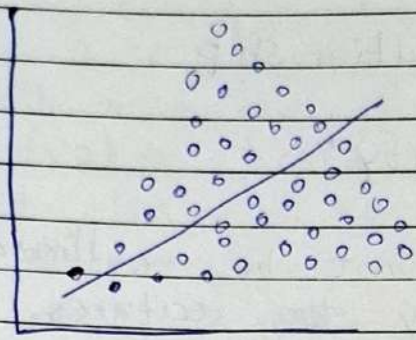


residuals should follow normal distribution and mean is near by 0.

4. Homosedasticity.

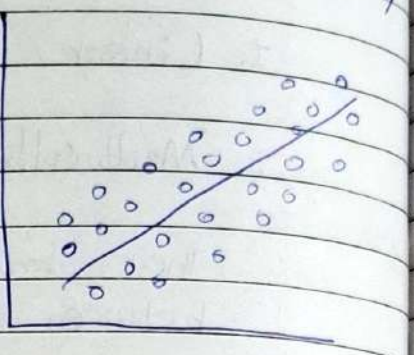
Variance of the residuals remains constant across all levels of the independent variables

Heteroscedasticity



X

Homoscedasticity



✓